Question Gate vs Direct LLM

An evidence-based audit of whether the question-classification layer should be removed, reduced, or kept

Scope	app/services/question_gate.py, app/services/transcript_quality.py, app/routes/audio_ws.py, app/core/config.py
Evidence base	8 real session logs (1,441 transcripts, 1,019 gate decisions, 465 answers); 104 rule-gate cases; 23 live tier-2 classifier calls; 28 live main-LLM calls on the unchanged production prompt; 19 end-to-end pipeline runs
Model under test	Answer: groq / openai/gpt-oss-120b | Tier-2: groq / openai/gpt-oss-20b
Code changed	**None.** This is an audit. Every test ran against the unchanged production code, prompt and settings.
Recommendation	**HYBRID APPROACH** — keep the rule gate, add the lightweight filter, fix three named defects. Do not remove the gate. See section 22.

1. Executive Summary

You asked whether the strict “is this a question?” step can be replaced by a small local filter that only rejects obvious acknowledgements, letting the main LLM work out meaning from conversation context. I inspected the code, measured the real gate on real session logs, and ran the actual model on the actual production prompt.

The answer is no — but not for the reason you would expect. The premise behind the proposal is that the gate is a slow, expensive, strict grammar check standing between the interviewer and the LLM. That is not what it is. The gate that decides 99.3% of your answered questions is a pure-CPU function that costs **0.1–1.8 milliseconds and zero tokens**. It is not on the latency path in any meaningful sense, and removing it would save nothing.

Meanwhile, what the gate actually protects you from is not bad grammar. It is **speech-to-text noise**. Your logs are full of it: *“Tarek, Mori, Mori.”*, *“Cine, Mosh, Bitu.”*, *“Cork, microservices, NgoDB, DioTool, DioTool, DioTool.”* I sent those straight to your production LLM with your production prompt. It did not say “that is not a question”. It **invented three projects that do not exist on the resume** and presented them as the candidate's own work, in the middle of a live interview.

> **The single most important measurement in this report**
>
> Simulating the proposed lightweight filter over your real session logs: it rejects only **8.1%** of utterances where the current gate rejects **56.1%**. **86.6% of everything the gate currently rejects would reach the main LLM.** Expensive LLM answer calls would go from 434 to 908 — **2.1×** — on the same audio. Your logs already show rate-limit stalls (first-token p99 of 15.8s, pipeline p99 of 25.9s). Doubling the call rate makes that materially worse.

What is genuinely wrong with the current gate

The proposal is pointing at something real, just not at the right layer. The rule gate is fine. The problems are concentrated in three specific, named places:

#	Defect	Evidence	Cost
D1	Tier-2 LLM classifier over-uses IGNORE	Live: *“Now the architecture part.”* → IGNORE. *“You know, I wanted to ask about RAG.”* → WAIT (then dropped). Logs: 235 IGNORE vs 3 ANSWER.	Real questions silently lost
D2	`"can you explain"` is treated as a complete follow-up	Pipeline run: *“Can you explain”* + *“RAG?”* produced **2 answers at every pause length tested (0.3s / 0.8s / 1.5s / 2.5s)**. Follow-ups bypass all waiting.	1 question → 2 wrong answers, 2 LLM calls
D3	Clause splitting damages spelled-out acronyms	*“Explain A.P.I. design.”* → the gate sends only *“Explain A.P.I.”*; *“We used MySQL... sorry, PostgreSQL.”* → sends *“sorry, PostgreSQL.”*	Part of the question thrown away

Measured rates (manual labelling, seeded random sample of 80 accepts + 80 rejects)

Metric	Current gate	Proposed (filter + LLM)	Note
False negatives (real content rejected)	**20.0% ±8.8pp of rejects**	**~2–3%**	Proposal genuinely wins here
of which: a real question was lost	**11.3% of rejects (~62 questions)**	**near zero**	Proposal wins
of which: scenario setup lost	**8.8% of rejects (~49)**	**becomes a premature answer**	Proposal is worse
False positives (should not have been answered)	**11.3% ±6.9pp of accepts (~49)**	**~47% of accepts (~429)**	Proposal loses badly
Gate latency on an answered question	**0.1–1.8 ms**	**~0.01 ms**	Tie in practice

Metric	Current gate	Proposed (filter + LLM)	Note
Expensive LLM calls per 988 utterances	**434**	**908 (+109%)**	Proposal loses
Total tokens per 988 utterances	**~929k**	**~1.53M (+64%)**	Proposal loses

Recommendation in one line

HYBRID APPROACH

Keep the tier-1 rule gate exactly as it is — it is free and it decides 99.3% of your answers correctly. Add the explicit lightweight non-content filter you proposed, but put it **in front of the tier-2 LLM classifier, not in front of the main LLM**, so it cuts classifier calls instead of removing the noise wall. Then fix D1, D2 and D3. That captures every real benefit the proposal was reaching for, without the 2.1× call increase and without letting hallucinated transcripts reach the answer model.

2. Current Architecture (as built, not as assumed)

The conceptual diagram in the brief has one box called “Question Gate”. The real system has **five** filtering stages before the LLM, and two of them already do exactly what the proposed “lightweight non-content filter” is meant to do.

```
Audio (system loopback - the interviewer's voice)
  |
  v
[1] VAD  --  app/services/vad.py
      segment_min_speech_ms=320   drops clicks/blips before they cost an STT call
      segment_end_silence_ms=420  decides when an utterance ended
      segment_max_seconds=8.0     force-cut, carries an utterance_id
  |
  v
[2] STT  --  app/services/stt/*  (Groq Whisper / NVIDIA / local)
  |
  v
[3] TRANSCRIPT QUALITY FILTER  --  app/services/transcript_quality.py
      assess_transcript(): looping_repetition | no_real_words | mostly_non_words
                           | silence_hallucination | low_confidence
      <-- THIS IS ALREADY THE 'LIGHTWEIGHT NON-CONTENT FILTER'
  |
  v
[4] BUFFER + DEBOUNCE  --  question_gate.QuestionAnswerPipeline
      coalesces fragments of one utterance; decides WHEN, not just WHETHER
  |
  v
[5] THE GATE
      tier 1: _classify_question()   pure CPU, 0 tokens, 0.1-1.8 ms   -> 99.3% of answers
      tier 2: _run_fast_intent_classifier()  gpt-oss-20b, ANSWER/WAIT/IGNORE
              only for reason == 'no_question_signal'                  -> 0.7% of answers
  |
  v
[6] MAIN LLM  --  _build_answer_messages() + gpt-oss-120b, streaming
  |
  v
Answer -> overlay
```

> **Finding: the proposed filter already exists**
>
> `transcript_quality.assess_transcript()` is the lightweight non-content filter, and it already sits between STT and the gate. It is **better** than the proposed word list because it is audio-aware: it only treats *“thank you”* as a hallucination when the VAD says there was under a second of voiced audio, or the decoder confidence was under 0.35. A pure word list cannot tell a real spoken “thank you” from a Whisper silence artefact. So the proposal is not “add a filter”. It is only “delete the gate”.

3. Current Question Gate Analysis

3.1 Where it is called and what it receives

Question you asked	Answer from the code
Where is the gate called?	`_process_after_debounce()` (question_gate.py:378) calls `_run_gate()` on every debounce cycle; `_generate_answer()` (line ~660) re-uses the precomputed result. Tier-2 is called from the same loop at line ~350.
What input does it receive?	A single string: the buffered transcript, i.e. all fragments of the current utterance joined by `_combine_transcript_parts()` with overlap removal. **No conversation history** for tier-1. Tier-2 gets the last 2 Q/A turns.
What does it check?	In order: empty → exact small-talk set → follow-up phrase set / dangling pronoun → split into clauses → per clause: interview intent (fuzzy) → credible '?' → question-word prefix → question phrase → X-vs-Y → bare tech keyword (terse text only).
Does it use an LLM?	**Tier-1: no.** Pure regex and word lists. **Tier-2: yes**, but only for the single reason `no_question_signal`, and it is skipped entirely while the VAD says the utterance is still open.
What happens after a rejection?	`_generate_answer()` broadcasts a `question_gate` event with should_answer=false and returns before the LLM. The buffer is either dropped at once (empty / small_talk / fast_intent_ignore) or held for `utterance_merge_gap_seconds`=4.0s in case a continuation arrives.
Are rejected transcripts stored?	**Partly.** Every transcript is broadcast as a `transcript` event and lands in the durable session log and on the overlay. But `_store.add_history()` is only reached after a successful answer, so a rejected utterance **never enters the conversation history the LLM sees**.
Is follow-up context available?	**Not to tier-1.** `_run_gate()` takes a history argument and ignores it (line 1065–1070). Tier-2 gets 2 turns. The main LLM gets 3 turns. So the gate decides 'is this answerable' blind to what was just discussed.

3.2 Latency and token cost, measured

Stage	Latency	Tokens	Share of answered questions
Tier-1 rule gate, short question (3 words)	`0.096 ms`	`0`	—
Tier-1 rule gate, 25-word scenario question	`5.0 ms`	`0`	—
Tier-1 rule gate, 150-word merged buffer	`29.7 ms`	`0`	—
Tier-1 verdict resolves the question	`<2 ms typical`	`0`	**431 / 434 = 99.3%**

Tier-2 LLM classifier (live, n=23)	`p50 455 ms`
	`p90 1,213 ms`
	`p95 1,215 ms`
	`max 1,218 ms`
		`~300–470 in`
		`up to 400 out`
			3 / 434 = 0.7%

Tier-2 timeout rate	`4/23 = 17.4% live` `~40% in logs`	`wasted`	—
Debounce before first evaluation	`30 ms settled` `180 ms otherwise`	`0`	all

This is the fact that decides the whole question

Tier-2 almost never delays a real answer. It only runs when tier-1 returned `no_question_signal`, and utterances in that bucket end up being answered just 0.7% of the time. So the ~455ms–1.2s classifier cost is paid almost entirely on utterances that were **never going to be answered anyway**. Removing the gate to "go faster" removes latency that real questions do not experience.

3.3 What reaches the expensive path today

Gate reason	Count	Share	Verdict
question_mark	329	33.3%	**answer**

Gate reason	Count	Share	Verdict
fast_intent_ignore	235	23.8%	reject
no_question_signal (tier-2 gave no verdict)	160	16.2%	reject
too_short	128	13.0%	reject
question_prefix	60	6.1%	answer
incomplete_fragment	27	2.7%	reject
interview_intent	19	1.9%	answer
keyword_match	9	0.9%	answer
follow_up	6	0.6%	answer
manual_override (/ask button)	6	0.6%	answer
fast_intent_wait	4	0.4%	reject
fast_intent_answer	3	0.3%	answer
question_phrase	1	0.1%	answer
interview_introduction	1	0.1%	answer

988 gate decisions from 8 real session logs (31 screen-analysis events excluded). **43.9% reach the expensive LLM path; 56.1% are rejected.**

4. Proposed Architecture

```
Audio -> VAD -> STT -> [lightweight non-content filter] -> MAIN LLM + history -> Answer

filter rejects ONLY: thank you / thanks / okay / ok / good / great / alright /
                     fine / um / uh / hmm / yeah / yep / right / sure ...
filter must NOT reject anything meaningful.
```

Two things follow immediately from the definition, and both matter more than they look:

- **The filter can only reject what it has a word for.** It is an exact-match list. It cannot reject *"Tarek, Mori, Mori."*, *"Cine, Mosh, Bitu."*, *"Rosh is a great thing to do with the file."* or *"Definitely have a role in the role."* — all verbatim from your logs.
- **The main LLM has no way to say "wait" or "nothing here".** It is an answer generator behind a system prompt that says *'Answer as yourself… never reply about the question instead of answering it… if ambiguous, answer the most likely reading rather than asking for clarification.'* Given noise, it answers. This is not a hypothesis — section 6 shows what it actually produced.

Simulation of the proposed filter over the real corpus

	Current	Proposed	Change
Utterances evaluated	988	988	—
Rejected before the LLM	554 (56.1%)	**80 (8.1%)**	–474
Sent to the main LLM	434 (43.9%)	**908 (91.9%)**	**+109%**
Of what the gate rejects today, how much would reach the LLM	—	**480 / 554 = 86.6%**	—

What would leak through, by the reason it is rejected today

fast_intent_ignore	229
no_question_signal	150
too_short	70
incomplete_fragment	27
fast_intent_wait	4

Verbatim examples of what would newly reach the answer model

```
'Eeeh, Eeeh, Eeeh, Eeeh, Eeeh.'
'Tensile Bucke, RZP, Czart, NgoDB, PSG, Tx4, GPT-5, PSG, RZP, Czart. Tch.'
'Cork, microservices, NgoDB, DioTool, DioTool, DioTool, DioTool, DioTool.'
'Kalsangal, I devaftee, hey, hey, hey.'
'Tarek, Mori, Mori.'          'Cine, Mosh, Bitu.'          'Tiroga.'
'Reacty, Reacty.'             'Corault.'                   'Ampro-Pic.'
'Tt.s.n.s.r.d.s.n.t.'         'Cangbu, buraday.'           'Catering.'
"I don't know what I'm doing."         'I will talk to you guys.'
'Definitely have a role in the role.'  'The tourism is basically up.'
"It's interesting, I can't see what he's going to do."
'Music'   'Wow.'   'Caution.'   'R'   'Not bad.'   'Tch.'
```

All verbatim from logs/default_*.jsonl. None of these are matched by the proposed word list.

5. Test Methodology

Test	What it does	Live?	n
T1 — rule-gate battery	Calls the unchanged `_classify_question()` on every case in the brief; records verdict, reason, focus, completeness, and per-call time over 50 repeats.	offline	104
T2 — tier-2 classifier	Calls the unchanged `_run_fast_intent_classifier()` against the real Groq endpoint with real .env settings; measures wall-clock latency per call.	**live**	28 (23 reached tier-2)
T3 — main LLM understanding	Builds messages with the unchanged `_build_answer_messages()` and the unchanged production system prompt, with a realistic resume + JD + history, and streams from gpt-oss-120b. Prompt was **not** modified.	**live**	28
T4 — proposed-filter simulation	Replays every gate decision in the 8 real session logs through a throwaway model of the proposed word-list filter and compares accept/reject.	offline	988
T5 — gate CPU benchmark	Times the rule gate on 3-word, 25-word and 150-word buffers over 200 repeats.	offline	600
T6 — end-to-end pipeline	Drives the real `QuestionAnswerPipeline` with simulated VAD speech-state and transcripts at controlled pause lengths; the answer LLM is stubbed so what is measured is purely gate + debounce + history.	offline	19 runs
T7 — manual labelling	Seeded random sample (seed 42) of 80 accepted and 80 rejected utterances from the logs, labelled by hand as correct / false negative / false positive.	manual	160

> **Honest limits of this evidence**
>
> T7 is single-rater labelling, so at n=80 the 95% confidence interval is roughly ±7–9 percentage points. The log corpus is 8 sessions from one deployment, not a cross-user sample. The T3 live LLM calls are single-sample per case at temperature 0.3, so individual answers will vary on a re-run — the pattern across 28 cases is the evidence, not any one answer. Latency figures from logs include periods where the account was rate-limited, which is itself a finding rather than noise.

6. Standalone Non-Content Tests

You asked me not to assume every short word is useless. I tested each one against the real gate, then asked whether ignoring it could ever lose something.

Utterance	Current gate	Safe to ignore?	Always?	Where does it belong?
thank you	`reject — small_talk`	**YES**	**YES**	local filter
thanks	`reject — small_talk`	**YES**	**YES**	local filter
okay / ok	`reject — small_talk`	**YES**	**YES**	local filter
good	`reject — too_short`	**YES**	**YES**	local filter
great	`reject — too_short`	**YES**	**YES**	local filter
alright	`reject — too_short`	**YES**	**YES**	local filter
fine	`reject — too_short`	**YES**	**YES**	local filter
yeah / yep	`reject — too_short`	**YES**	**YES**	local filter
hmm / um / uh	`reject — too_short`	**YES**	**YES**	local filter
right	`reject — too_short`	**YES**	**see 6.1**	local filter
sure	`reject — too_short`	**YES**	**YES**	local filter

6.1 Does ignoring these ever cause a problem? — tested, not assumed

This is the interesting part. In a normal assistant, dropping an acknowledgement would corrupt the conversation history. In **this** system it cannot, and I verified why in the code and then end to end.

- `_store.add_history()` is only reached after a successful answer (question_gate.py:~838). A rejected utterance never enters history, so the LLM's view of the conversation is a clean list of question/answer pairs with no acknowledgements in it either way.
- **Scenario 5 run end to end** (‘Can you explain RAG?’ → ‘Okay.’ → ‘Why is it useful?’): history after the run contained exactly 2 turns — `What is RAG?` and `Why is it useful?`. The dropped ‘Okay.’ caused **zero** context loss.
- Better than that: the pipeline actually buffered ‘Okay. Why is it useful?’ as one string and the gate's focus selection isolated `"Why is it useful?"` before sending it. The acknowledgement was stripped, not lost and not answered.

> **Verdict on section 3 of the brief**
>
> Yes, these are safe to ignore, and yes it is safe *always* in this specific architecture, because rejected utterances never touch conversation history. They should be handled by a local filter and should not reach the LLM. **The current gate already does this correctly for all 15 of them.** There is no bug here to fix and no benefit available from changing it.

7. Context-Dependent Words

You were right to separate these out. They are not the same class as ‘thank you’.

Word	Current gate	Is meaning context-dependent?	Would ignoring lose information?
why / why?	**ANSWER — follow_up**	Yes — refers to the last answer	**Yes — and the gate correctly answers it**
how / how?	**ANSWER — follow_up**	Yes	**Yes — gate answers it**
which one	**ANSWER — follow_up**	Yes	**Yes — gate answers it**
yes	`reject — small_talk`	Yes — confirms a premise	**Almost never in this app — see below**
no	`reject — small_talk`	Yes	**Almost never — see below**
right	`reject — too_short`	Yes — can be a tag question	**Rarely**
exactly	`reject — too_short`	Yes	**Rarely**
that / this	`reject — incomplete_fragment`	Yes	**Rarely, alone**
it	`reject — too_short`	Yes	**Rarely, alone**

7.1 The “Did you use RAG?” → “Yes.” case, tested directly

Your example asks whether ignoring a bare ‘Yes.’ loses conversational information. I tested it two ways.

- **Whose voice is it?** This app captures system loopback audio — the interviewer. The candidate's own ‘Yes’ is not in the input stream at all. So the case is ‘the interviewer said Yes’, which in an interview means ‘correct, carry on’ — an acknowledgement, not a question.
- **What does the LLM do if you send it anyway?** Live test S23: history contained *“Did you use RAG in your project?”* / *“Yes — the document assistant is a RAG pipeline over ChromaDB.”*; I sent `"Yes."`. The model replied, in full: **`“Yes.”`** One token. Correct behaviour, but worthless on screen, and it cost a full 1,402-token prompt to produce.

> **Verdict**
>
> Ignoring a bare ‘Yes’ / ‘No’ loses no information the system can act on, because in this app they are the interviewer's acknowledgement rather than an answer the candidate needs. And the one case where sending it to the LLM is harmless (‘Yes.’ → ‘Yes.’) still burns a full prompt. **Both architectures reach the same place; the current one gets there for free.**

8. Follow-Up Tests (Scenarios 1–5, run end to end)

Each scenario was driven through the real pipeline with simulated VAD speech-state and real pauses. The gate decision, what was sent to the LLM, and the stored history were all recorded.

Scenario	Turn 2	Gate verdict	Sent to LLM	History	Result
SC1	“Why would you use it?”	`ANSWER follow_up`	rewritten to name RAG + the prior answer	2 clean turns	**correct**
SC2	“Why?”	`ANSWER follow_up`	rewritten to name the DB question + answer	2 clean turns	**correct**
SC3	“What challenges did you face?”	`ANSWER question_mark`	raw + 3 turns of history	2 clean turns	**correct**
SC4	“What about Django?”	`ANSWER question_mark`	raw + 3 turns of history	2 clean turns	**correct**
SC5	“Okay.” then “Why is it useful?”	`ANSWER question_mark`	focus isolated `"Why is it useful?"`, dropping ‘Okay.’	2 clean turns	**correct**

8.1 Would the direct-LLM path do better? — live A/B on the same model

I ran each follow-up twice against gpt-oss-120b with the unchanged production prompt: once as the raw words with history in the prompt (what the proposed architecture would do), and once through the current gate's `_resolve_followup()` rewrite.

Case	Raw text + history (proposed)	Gate rewrite (current)	Better
“Why would you use it?” after ‘What is RAG?’	**Correct — 5 reasons to use RAG**	**Correct — 5 reasons to use RAG**	tie
“Why?” after ‘What database did you use?’	**Correct, but drifted — started on PostgreSQL then wandered into FastAPI, Docker, ECS and CI/CD**	**Correct and on-topic — stayed on PostgreSQL and Redis, 2 sentences**	**current**
“What challenges did you face?”	**Correct — Django project challenges**	n/a (not a follow_up)	tie
“What about Django?”	**Correct — Django, framed against the FastAPI answer**	n/a (not a follow_up)	tie
“Why is it useful?” after ‘Can you explain RAG?’	**Correct — why RAG is useful**	n/a (question_mark path)	tie

> **Finding: the LLM does handle follow-ups from history — and the gate still adds value**
>
> The main model resolves *it*, *that*, and a bare *Why?* correctly from three turns of history. So the proposal's core claim is **true** for follow-ups. But the one measurable difference went the current architecture's way: on the bare “Why?”, the raw version drifted off the database question into four unrelated topics, while the gate's rewrite — which explicitly says *‘this is about what you JUST said, not a new topic’* — stayed tight. In a live interview the candidate reads this aloud, so drift is a real cost.

9. Miscommunication and Technical-Terminology Tests

Every row below is a live call to gpt-oss-120b with the unchanged production prompt and a resume that mentions RAG, LangChain, ChromaDB, FastAPI, Django, PostgreSQL, Redis, AWS and Docker.

Intended	STT transcript	Current gate	What the LLM understood	Correct?
Redis	“What is **red is** used for in your backend?”	`ANSWER question_mark`	Redis — caching, Celery broker	**YES**
SQL	“How comfortable are you with **sequel**?”	`ANSWER question_mark`	SQL — joins, window functions, CTEs	**YES**
AWS	“Have you deployed anything on **A B S**?”	`ANSWER question_mark`	AWS — ECS, ECR, RDS, CloudWatch	**YES**
Django	“Tell me about **jango**.”	`ANSWER question_prefix`	Django — DRF, ORM, admin, Celery	**YES**
LLM	“What is an **L.M.** and how does it work?”	`ANSWER question_mark`	large language model, next-token prediction	**YES**
API	“Explain **A.P.I.** design.”	**ANSWER, but focus truncated to ‘Explain A.P.I.’**	API design — but the word ‘design’ was never sent, so the answer is about what an API *is*, not how to design one	**partly**
FastAPI	“What is **fast API**?”	`ANSWER question_mark`	FastAPI	**YES**
RAG	“Can you give me an idea about **Rack**?” *(prior turn was about RAG)*	`ANSWER question_mark`	**Ruby Rack — ‘a minimal Ruby interface between web servers and frameworks’. It did not recover RAG from history.**	**NO**
RAG	“Can you give me an idea about **Rack**?” *(no prior context)*	`ANSWER question_mark`	**Fabricated — ‘Rack is a lightweight Python library that lets you spin up a simple HTTP server…’. No such library.**	**NO**

10. The RAG / Rack Test, in detail

> **This is the test that most directly contradicts the proposal's premise**
>
> The proposal rests on the idea that the main LLM can recover intended meaning from conversation context, so a strict gate is unnecessary. On the exact case you named — ‘Rack’ for RAG, **with the previous turn explicitly about RAG in the history** — it did not. It answered about Ruby Rack, confidently, and even added *‘I haven't used Rack directly’*, which is the worst possible thing for the candidate to read aloud two seconds after being asked about their own RAG pipeline.

But be careful about what this proves

It does **not** prove the gate is helping here. The gate passed ‘Rack’ through in both directions — it saw a credible question mark and a question prefix and said ANSWER. The failure is entirely downstream, in the answer model's disambiguation.

- **Current architecture on this case:** gate says ANSWER → LLM answers about Ruby Rack. Wrong.
- **Proposed architecture on this case:** filter passes it → LLM answers about Ruby Rack. Identically wrong.
- **Conclusion: this dimension is a tie.** Removing the gate neither fixes nor worsens misrecognised technical terms. Fixing it needs something neither architecture has — a session vocabulary built from the resume/JD that biases interpretation. That is a separate change and is listed in section 23.

Where the gate does help with misrecognition

One real mechanism exists and is worth keeping: when tier-1 matches an interview intent but the words came through badly (`_lexical_word_ratio < 0.75`), `_resolve_interview_intent()` tells the LLM what was *meant* instead of handing it the garbled text. Log evidence of this working: *“So, Okay, so they start your interview. Firstly, tell you about yourself. Ct.”* and *“Siddhubh.com. Okay, so let's start with the introduction. Tell me about yourself.”* both resolved to the clean intent. A direct-LLM path has no equivalent.

11. Self-Correction Tests

Utterance	Current gate	What is sent to the LLM	LLM output	Correct?
“I used Flask, actually, Django.”	**ANSWER — keyword_match**	whole clause, rewritten by `_resolve_keyword_mention()` into ‘talk about Flask’	answers about the **first** term	**NO**
“We used MySQL... sorry, PostgreSQL.”	**ANSWER — keyword_match, focus mangled to** `‘sorry, PostgreSQL.’`	`"sorry, PostgreSQL."`	PostgreSQL — right topic, but only by luck of which clause survived	**partly**
“The project was in 2023... no, 2024.”	`reject — no_question_signal, then tier-2 IGNORE`	nothing	n/a — dropped	**correct to drop**
“We used RAG... I mean Retrieval Augmented Generation.”	**ANSWER — keyword_match**	rewritten to ‘talk about rag’	answers about RAG	**YES**
“Tell me about Flask. Actually, tell me about Django.”	**ANSWER — question_prefix, focus =** `‘Tell me about Flask. tell me about Django.’`	both clauses	**Django only** — the model resolved the correction itself	**YES**
“What is MySQL? Sorry, I mean PostgreSQL. What is it good at?”	**ANSWER — question_mark**	whole thing	**PostgreSQL only** — correction resolved	**YES**

> **Finding: the LLM is good at corrections; the gate's clause splitting is not**
>
> When the whole utterance reaches the model, it handles self-correction cleanly — it answered Django, not Flask, and PostgreSQL, not MySQL, without being told to. The failures are caused by the gate **splitting the utterance and sending only part of it**: `"We used MySQL... sorry, PostgreSQL."` becomes `"sorry, PostgreSQL."`. **This is defect D3, and it argues for narrowing the gate's focus-selection, not for deleting the gate.**

Note also that a self-correcting **statement** from the interviewer (“*I used Flask, actually Django*”) is not a question at all, and the `keyword_match` rule answering it is a false positive in both architectures. See section 13.

12. Incomplete Question Tests

Each fragment pair was run through the real pipeline at four different pause lengths, with the VAD reporting speech start/stop realistically. The number in each cell is **how many answers the system produced for one question**. It should always be 1.

Question, split at the pause	0.3s	0.8s	1.5s	2.5s
“Can you explain” + “RAG?”	2	2	2	2
“What is the difference between” + “FastAPI and Django?”	1	1	1	2
“Why did you” + “choose Redis?”	1	1	1	2

12.1 Defect D2 — “Can you explain…” always double-fires

This is a genuine bug, reproducible at every pause length including 300ms. The cause is exact and sits in two places:

```
question_gate.py:1649  _is_followup_question()
    followups = { ... 'can you explain', 'can you elaborate', 'explain that', ... }
    -> 'Can you explain' matches EXACTLY, so reason = 'follow_up'

question_gate.py:1809  _should_keep_waiting()
    if gate.reason == 'follow_up':
        return False          # follow-ups are short by design; never wait

Net effect: the most common way an interviewer starts a sentence is classified as a
COMPLETE follow-up and fires instantly, bypassing question_soft_wait_seconds entirely.
```

The other two fragments behave correctly because they fall through to `question_prefix` with `complete=False`, which earns the 1.5s soft wait. At 2.5s they split — that is the documented and deliberate limit of `question_soft_wait_seconds`, not a bug.

12.2 Would the proposed architecture do better here?

No — it would be strictly worse. The proposed filter has no concept of waiting at all. Every fragment goes straight to the LLM the moment STT returns it. I tested what the model does with a bare fragment:

Sent	gpt-oss-120b response (live, unchanged prompt)
`"Can you explain"`	**Confidently explained the entire RAG assistant — chunking, embeddings, ChromaDB, Celery, ECS — a 5-bullet answer to a question that was never asked. Then ‘RAG?’ arrives and it answers again.**

> **The gate's real job is timing, not grammar**
>
> “Is this a question?” is only half of what the gate decides. The other half is **“is it finished?”** — `complete`, `_trails_off()`, `_looks_finished()`, `question_soft_wait_seconds`, `awaiting_more_of_utterance`. An answer-generating LLM has no way to express ‘wait’. Delete the gate and you delete the only component that can hold a half-spoken question.

13. Context Tests: natural speech, statements, and mixed utterances

13.1 Natural human speech (section 9 of the brief)

Utterance	Current gate	Correct?
“Um, can you explain RAG?”	`ANSWER — question_mark`	**YES**
“So basically, what is RAG?”	`ANSWER — question_mark`	**YES**
“Actually, yeah, can you tell me about RAG?”	`ANSWER — question_mark`	**YES**
“Can you, uh, explain RAG?”	`ANSWER — question_mark`	**YES**
“What is RAG, like, in simple terms?”	`ANSWER — question_mark`	**YES**
“You know, I wanted to ask about RAG.”	**tier-1 reject → tier-2 says WAIT → dropped after 4s**	**NO**

Five of six handled correctly by the free tier-1 rules. The sixth is a real miss, and it is a **tier-2** miss, not a tier-1 one — which is exactly where the fix belongs.

13.2 Statements that carry useful context (section 10 of the brief)

You called this critical, and you were right — but the answer is the opposite of what the brief expects. The gate does not discard these. It **answers** them, which is worse.

Statement	Current gate	What happens	Verdict
“I worked on a Django project.”	**ANSWER — keyword_match**	Rewritten to ‘talk about django’ and answered in full	**false positive**
“My backend was FastAPI.”	**ANSWER — keyword_match**	Answered as if asked about FastAPI	**false positive**
“We used PostgreSQL.”	**ANSWER — keyword_match**	Answered as if asked about PostgreSQL	**false positive**
“The main problem was data quality.”	`reject — no_question_signal`	Held 4s for a continuation, then dropped	**acceptable**
“I used RAG for document retrieval.”	`reject → tier-2 IGNORE`	Dropped	**acceptable**
“A production Linux server suddenly reaches 95–100% CPU.”	`reject → tier-2 IGNORE`	**Dropped** — but if the real question follows within 4s it is merged and the setup survives (verified end to end, SC6)	**fragile**

Is context lost? Not in the way the brief assumes. Setup sentences are held for `utterance_merge_gap_seconds` = 4.0s specifically so a scenario question can be built across sentences, and `_select_focus()` grows the answer text outward over neighbouring real sentences. I verified this end to end: ‘A production Linux server suddenly reaches 95 to 100% CPU.’ followed by ‘How would you troubleshoot it?’ produced **one** answer containing **both** sentences. The setup is only lost when the gap exceeds 4 seconds.

What the proposed architecture does to these: it sends the setup sentence to the LLM on its own. Live test S19 — I sent exactly that CPU sentence with no question attached, and the model produced a complete five-bullet troubleshooting answer immediately. Then the real question arrives and it answers again. **One question, two answers, two LLM calls, and the first one is on screen while the interviewer is still talking.**

13.3 Mixed utterances (section 11 of the brief)

Your concern was that a filter must not reject these just because they start with ‘okay’ or ‘yes’. The current gate already handles this correctly via `_strip_filler_lead()`, which loops until no leading filler remains.

Utterance	Current gate	Correct?
“Okay, but why did you use RAG?”	`ANSWER — question_mark`	**YES**
“Yes, can you explain that?”	`ANSWER — question_mark`	**YES**
“Right, so what was the main challenge?”	`ANSWER — question_mark`	**YES**
“Good, and what database did you use?”	`ANSWER — question_mark`	**YES**
“Okay, actually I wanted to ask about Redis.”	**tier-1 reject → tier-2 WAIT → dropped**	**NO**

4 of 5 correct at zero cost. The failure is the same shape as 13.1: an **indirect** request (‘I wanted to ask about X’) rather than a direct question. That phrasing is not in `_QUESTION_PREFIXES`, and tier-2 mishandles it. This is defect D1.

14. False Negative Analysis

Method: seeded random sample (seed 42) of 80 of the 554 rejected utterances from the real logs, labelled by hand. A **hard false negative** is a real question or request that was lost. A **soft false negative** is scenario setup that was dropped, degrading the answer when the question arrives.

Category	Count / 80	Rate	Extrapolated to 554 rejects
Correctly rejected (noise, filler, chatter, fragments)	64	**80.0%**	~443
Hard false negative — a real question lost	9	**11.3% ±6.9pp**	~62
Soft false negative — scenario setup lost	7	**8.8%**	~49
Total false negatives	16	**20.0% ±8.8pp**	~111

As a share of everything: ~111 of 988 utterances = **11.2% of all traffic is content that was wrongly dropped**. Against the 434 questions that were answered, the ~62 hard misses give a **missed-question rate of roughly 12.5%** — about one real question in eight is never answered.

14.1 The actual misses, verbatim

Rejected as	Utterance	What was lost
no_question_signal	“Okay, so let's start with your introduction. Thank you. Okay. Thank you.”	the opening request of the interview
no_question_signal	“And for what it is used”	a follow-up: ‘and what is it used for’
no_question_signal	“Okay, Vishal, that's really interested. So you work with the fastest. P.P. So please explain me any project that you've”	‘please explain me any project’
fast_intent_ignore	“So, Okay, so they start your interview. Firstly, tell you about yourself. Ct.”	‘tell you about yourself’
fast_intent_ignore	“Ct. I said what is FastAPI.”	a repeated question
fast_intent_ignore	“Correct. Let's introduce about yourself. Thank you.”	the introduction request
no_question_signal	“Thank you. Tell you about some projects that you worked on.”	a project request
no_question_signal	“Giu Mio Callu. for calculator app in Python.”	‘give me code for a calculator app in Python’
fast_intent_ignore	“Ape developed. Development page developer is not accepting bug. How to… Reson this issue.”	‘how to resolve this issue’
fast_intent_ignore	“So your application has 1 lakh users and your monthly OpenAI LLM bill has suddenly increased significantly.”	**scenario setup — the numbers the answer must reason from**
fast_intent_ignore	“The company now wants to scale it to 10 million documents using the same architecture, same vector database.”	**scenario setup**
no_question_signal	“Partition is 100% full.” / “Your platform receives repeated prompts.”	**scenario setup**

14.2 Which architecture fixes these?

Failure class	Current	Direct LLM	Filter + LLM
Real question buried in noise (9 cases)	**lost**	**recovered**	**recovered**
Indirect request (‘I wanted to ask about X’)	**lost**	**recovered**	**recovered**
Scenario setup (7 cases)	**lost if >4s gap**	**answered prematurely, then answered again**	**same**

> **The honest read on false negatives**

> **This is the proposal's strongest ground and it is genuinely strong.** Roughly one real question in eight is currently dropped, and most of those would be recovered by simply letting the utterance reach the model. But note where the misses come from: **almost every one of them is a tier-2 IGNORE or a tier-2 no-verdict**, not a tier-1 rule failure. You do not need to delete the gate to fix them — you need to fix the one tier that is already an LLM.

15. False Positive Analysis

Same method: seeded random sample of 80 of the 434 accepted utterances, labelled by hand. A false positive is an utterance that reached the expensive LLM path when it should not have.

Category	Count / 80	Rate	Extrapolated to 434 accepts
Correctly answered	71	**88.8%**	~385
False positive — noise or non-content answered	9	**11.3% ±6.9pp**	~49
of which pure garbage answered	6	7.5%	~33

15.1 The false positives, verbatim

Accepted as	Utterance	Why it is wrong
question_mark	“Cora? Cora?”	STT noise that happens to carry a question mark
question_mark	“Correct?”	an acknowledgement with rising intonation
question_mark	“Und Hager Follow-up studieren. ■■■■■■ ■■■■■■ ■■■■ ■■■■?”	cross-language STT garbage
question_mark	“We are in some of the Could you repeat the question?”	the interviewer asking the candidate to repeat — must not be answered
keyword_match	“Frappe, Frappe, framework.”	a bare mention, not a question
keyword_match	“means LangChain output parser.”	a sentence fragment
keyword_match	“GENAI-GraphQL, M.A.”	garble that happens to contain a keyword
question_prefix	“How would you prevent” / “What kind of states did you”	truncated fragments fired before the rest arrived

15.2 The same measurement for the proposed architecture

Every one of the 474 extra utterances the proposed filter would pass through is, by definition, something the current system rejected. From the labelling above, roughly 80% of the rejected pool is genuine noise or filler.

	Current	Proposed (filter + LLM)
Utterances reaching the answer LLM	434	908
Of those, unnecessary	**~49 (11.3%)**	**~429 (47%)**
Unnecessary LLM calls per 988 utterances	**~49**	**~429 — a 8.8× increase**

15.3 What an unnecessary call actually produces — live

This is the part that turns a cost problem into a correctness problem. I sent noise from your own logs to your production model with your production prompt.

Sent	What appeared on the overlay
`"Cine, Mosh, Bitu."`	**“Cine is a video-streaming microservice I built using FastAPI… Mosh is the RAG assistant pipeline… Bitu is the Celery-driven background worker…” Three projects invented from nothing and attributed to the candidate.**
`"Tarek, Mori, Mori."`	**“Sure, I'm comfortable building RAG pipelines with LangChain…” — a confident answer to no question at all.**
`"Thank you."`	**A full five-bullet ‘tell me about yourself’ answer.**
`"Okay."`	**A five-bullet scaling and infrastructure answer to a question nobody asked.**

Why this is the decisive finding

The system prompt tells the model *'never reply about the question instead of answering it'* and *'if ambiguous, answer the most likely reading rather than asking for clarification'*. Those rules are correct for real questions and catastrophic for noise. There is nothing in the proposed architecture between STT hallucination and a fabricated project on the candidate's screen mid-interview. **The current gate is the only thing standing there.**

16. Latency Analysis

16.1 Measured gate latency

Measurement	p50	p90	p95	p99	max
Tier-1 rule gate (per call, by buffer size)	0.10 ms 3 words	5.0 ms 25 words	—	—	29.7 ms 150 words
Tier-2 classifier, live (n=23)	455 ms	1,213 ms	1,215 ms	—	1,218 ms
Debounce before first evaluation	30 ms settled	180 ms	—	—	—

16.2 End-to-end latency from the real session logs (n as shown)

Metric	n	p50	p90	p95	p99	max
STT round trip (ms)	181	328	809	1,271	4,265	5,797
Time since last spoken fragment (ms)	76	185	195	681	1,673	1,673
Pipeline: first word heard → LLM call (ms)	465	503	3,529	5,162	**25,890**	**125,818**
LLM first-token latency (ms)	77	720	2,872	5,180	**15,810**	**15,810**
Full answer duration (ms)	79	988	4,935	9,768	15,828	15,828

The tail is the story. A p50 first-token of 720ms with a p99 of 15.8s is not a model that is sometimes slow — that is **rate limiting**. The code's own comments record measuring 8,000 TPM from Groq's `x-ratelimit-limit-tokens` header for gpt-oss-120b, and I reproduced the same pattern live during this audit: repeated first-token times clustering at 8.5–10.9s on back-to-back calls.

16.3 Latency under each architecture

Path	Current	Proposed	Winner
A question tier-1 recognises (99.3% of answers)	0.1–1.8 ms + 30 ms debounce	~0.01 ms + 0 debounce	**proposed, by ~30 ms**
A question only tier-2 recognises (0.7%)	+455 ms p50 / 1,213 ms p90	0 ms	**proposed**
An utterance that is never answered	+455 ms p50, invisible to the user	—	tie (invisible)
Queueing behind other calls at 8,000 TPM	434 answer calls	**908 answer calls — 2.1× the TPM pressure**	**current, decisively**

> **Net latency verdict**
>
> The proposed architecture saves about **30 milliseconds** on a typical question, and throws away the debounce that stops one question becoming two answers. Against that it doubles the load on a token bucket that is **already producing 15.8-second stalls**. On measured evidence the proposal is **slower**, not faster, in exactly the conditions that matter.

17. Token Analysis

Component	Tokens per call	Calls per 988 utterances	Total
Tier-1 rule gate	0	~988	0
Tier-2 classifier	~350 in + ~150 out (budget 400 out)	~400	~200,000

Component	Tokens per call	Calls per 988 utterances	Total
Main answer call	`~1,450 in + ~230 out (system prompt alone = 1,216)`	`434`	`~729,000`
Current total		**`834 model calls`**	**`~929,000`**
Lightweight filter	`0`	`988`	`0`
Main answer call	`~1,680`	**908**	**~1,525,000**
Proposed total		**`908 model calls`**	**~1,525,000**

Comparison	Change
Total model calls	**834 → 908 (+9%)**
Calls to the **expensive** model	**434 → 908 (+109%)**
Expensive-model tokens	**729k → 1,525k (+109%)**
Total tokens	**929k → 1,525k (+64%)**
Answers per minute possible at 8,000 TPM	**~4.8 → the same, but competing with 2.1× the demand**

The saving from deleting tier-2 (~200k tokens on a small, cheap model) is real but is dwarfed by the cost of 474 extra calls to the large model (~796k tokens). **The trade is roughly 1 token saved for every 4 tokens added, and the added tokens are on the model that is already the bottleneck.**

18. Main LLM Understanding Analysis

The central question of the audit: given conversation history and the unchanged production prompt, can gpt-oss-120b already understand these cases without a classification step? 28 live calls, no prompt changes.

Capability	Result	Evidence
Resolve a pronoun follow-up ('why would you use **it**')	**PASS**	S1 — correctly answered about RAG from history alone
Resolve a bare 'Why?'	**PASS with drift**	S2 — correct topic, but wandered into FastAPI/Docker/CI. The gate's rewrite (S26) stayed on topic
Carry a topic across turns ('What challenges…')	**PASS**	S3
Comparative follow-up ('What about Django?')	**PASS**	S4
Survive a dropped acknowledgement	**PASS**	S5 — no context loss
Recover a mis-transcribed common term	**PASS**	S7–S12: red is→Redis, sequel→SQL, A B S→AWS, jango→Django, L.M.→LLM, A.P.I.→API
Recover a mis-transcribed term **from history**	**FAIL**	S6 — 'Rack' with RAG in history → answered about Ruby Rack
Handle a self-correction inside a question	**PASS**	S13, S14 — answered Django not Flask; PostgreSQL not MySQL
Recognise an incomplete fragment and wait	**FAIL — structurally impossible**	S16 — 'Can you explain' produced a full confident answer on a guessed topic
Recognise a scenario setup and wait	**FAIL — structurally impossible**	S19 — answered the CPU scenario before the question was asked
Recognise a statement as not a question	**FAIL**	S18 — 'I worked on a Django project.' answered as a Django question
Recognise pure non-content and stay silent	**FAIL**	S21 'Thank you.' and S22 'Okay.' both produced full 5-bullet answers
Recognise STT noise and stay silent	**FAIL — severe**	S24, S25 — fabricated projects and experience
Handle a bare 'Yes.' gracefully	**PASS**	S23 — replied 'Yes.'

Why the LLM fails the last four rows — and why no prompt change fixes it cheaply

You asked me to document the reason without fixing it. The reason is structural, not a prompt weakness. The answer path is a **streaming generator**: the moment it is called, `answer_start` is broadcast, tokens stream to the overlay, and `add_history()` records the turn. There is no return value that can mean *'not yet'* or *'nothing here'*. Even if the prompt were changed to allow the model to decline, you would still pay the full call, the full latency and the full token cost **before** learning that the utterance was noise — and the overlay would flash a started answer that then had to be retracted.

19. Answer Quality Analysis

Case	Right meaning?	Right context?	Natural / speakable?	Too verbose?	Repeats?
Follow-up, gate rewrite (S26)	**YES**	**YES**	**YES**	**no — 2 sentences**	**no**
Follow-up, raw + history (S2)	**YES**	**YES**	**YES**	**yes — 5 bullets, drifted**	**partly**

Case	Right meaning?	Right context?	Natural / speakable?	Too verbose?	Repeats?
Misheard term, common (S7–S11)	YES	YES	YES	no	no
Misheard term, RAG→Rack (S6)	NO	NO	YES	no	no
Self-correction (S13, S14)	YES	YES	YES	borderline	no
Gate-truncated correction (S15)	by luck	partly	YES	yes	no
Fragment answered early (S16)	NO	NO	YES	yes	will repeat
Setup answered early (S19)	NO	PARTLY	YES	yes	will repeat
Non-content (S21, S22)	NO	NO	YES	yes	yes
Noise (S24, S25)	NO	NO	YES	yes	fabricates

A pattern worth naming: **the answers are always fluent and always confident, whatever the input.** The prompt is doing its job — it produces natural, speakable text. That is exactly why the input to it has to be filtered. Fluency on garbage is worse than a visible error, because the candidate reads it aloud believing it.

20. Decision Matrix

Scenario	Current Gate	Direct LLM (no filter)	Lightweight Filter + LLM	Winner
“What is RAG?”	**ANSWER, 0.1 ms, 0 tokens**	**correct**	**correct**	tie
“Why?”	**ANSWER — follow_up, rewritten, stayed on topic**	**correct but drifted to 4 unrelated topics**	**same drift**	**Current**
“What about Django?”	**ANSWER — question_mark**	**correct**	**correct**	tie
“Okay”	**reject, free**	**full 5-bullet scaling answer**	**rejected by word list**	tie (Current / Filter)
“Thank you”	**reject, free**	**full ‘tell me about yourself’ answer**	**rejected by word list**	tie (Current / Filter)
“Okay, why?”	**ANSWER — filler stripped**	**correct**	**correct**	tie
“Rack” meaning RAG	**ANSWER, but LLM answers Ruby Rack**	**same wrong answer**	**same wrong answer**	tie — **neither fixes it**
Self-correction ‘MySQL… sorry, PostgreSQL’	**focus truncated to ‘sorry, PostgreSQL.’**	**whole utterance → correct**	**correct**	**Filter + LLM**
Incomplete ‘Can you explain…’	**fires immediately → 2 answers (defect D2)**	**fires immediately → 2 answers, always**	**same**	**none** — both broken
Incomplete ‘Why did you…’	**waits 1.5 s, merges correctly**	**answers the fragment, then answers again**	**same**	**Current**
Scenario setup then question	**merges within 4 s → one answer with the facts**	**two answers; the first ignores the question**	**same**	**Current**
Real question buried in noise	**often dropped (defect D1)**	**recovered**	**recovered**	**Filter + LLM**
STT noise (‘Cine, Mosh, Bitu.’)	**reject, free**	**invents three projects**	**invents three projects — word list misses it**	**Current, decisively**

Tally	Current wins	Filter + LLM wins	Tie / neither
13 scenarios	**4**	**2**	7

21. Final Architecture Comparison

#	Question	Answer	Evidence
1	Is the proposed architecture faster?	**No**	Saves ~30 ms of debounce and a 0.1–1.8 ms gate on the answered path, but doubles calls into a token bucket already producing a 15.8 s p99 first-token
2	Is it more accurate?	**No, net**	Fixes ~62 missed questions; creates ~380 new unnecessary answers, some of them fabrications
3	Is it better for follow-ups?	**No — marginally worse**	The LLM does resolve follow-ups from history, but the bare ‘Why?’ drifted without the gate's rewrite
4	Is it better for miscommunication?	Tie	Both architectures answered ‘Rack’ as Ruby Rack. Neither helps
5	Is it better for imperfect pronunciation?	Tie	6 of 7 mis-transcribed terms already pass the gate untouched and are recovered by the LLM
6	Does it reduce unnecessary LLM calls?	**No — the opposite**	434 → 908 calls; unnecessary calls ~49 → ~429

#	Question	Answer	Evidence
7	Does it increase LLM token usage?	**Yes**	929k → 1.53M per 988 utterances (+64%)
8	Does it increase rate-limit risk?	**Yes, substantially**	2.1× the demand on an 8,000 TPM budget already being exceeded
9	Does it create new failure modes?	**Yes — three**	(a) noise becomes fabricated resume content; (b) every fragment answered twice; (c) scenario setups answered before the question
10	Is the added complexity worth it?	**The complexity is not the issue**	The proposal is *simpler*. It is simpler in the wrong place — it removes the timing logic, which is the half of the gate that has no LLM substitute

22. Risk Analysis

Risk	If you REMOVE the gate	If you KEEP it as-is	Under the HYBRID
STT noise answered as real content	**High — demonstrated live**	**Low**	**Low**
Fabricated projects on screen mid-interview	**High — demonstrated live**	**Low**	**Low**
Real questions missed	**Low**	**~12.5% of questions**	**Medium → Low after D1 fix**
One question answered twice	**High — every fragment**	**Medium (D2)**	**Low after D2 fix**
Rate-limit stalls / 429s	**High — 2.1× demand**	**Already happening**	**Improved — fewer tier-2 calls**
Cost	**+64% tokens**	**Baseline**	**Slightly below baseline**
Part of a question thrown away by focus selection	**None — no focus logic**	**Medium (D3)**	**Low after D3 fix**
Scenario setups lost or answered early	**High**	**Medium**	**Medium**

23. Final Recommendation

> **HYBRID APPROACH**
>
> Keep the tier-1 rule gate. Add the lightweight non-content filter, but place it **before the tier-2 classifier** rather than before the main LLM. Fix defects D1, D2 and D3. Do not let unclassified transcripts reach the answer model.

Why

- **The premise does not survive measurement.** The gate is not a slow, strict grammar check. The part that decides 99.3% of your answers costs 0.1–1.8 ms and zero tokens. There is no latency or cost to reclaim by deleting it.
- **'Is this a question?' is only half of what it does.** The other half is 'is it finished, and is there anything here at all?' An answer-generating LLM has no way to express *wait* or *nothing*. Delete the gate and you delete the only component that can hold a half-spoken question or refuse an empty one.
- **The noise problem is bigger than the grammar problem.**86.6% of what the gate rejects would pass your proposed word list, because it is mangled STT output, not acknowledgements. Sent to the model, that output produced invented projects attributed to the candidate.
- **But your instinct about the failures is correct.** About one real question in eight is being dropped, and the drops are concentrated in tier-2, the one tier that is already an LLM. That is where to spend the effort — not on deleting the free tier that is working.

Expected impact of the hybrid

Dimension	Expected change	Confidence
Latency, answered path	Unchanged (±2 ms)	**high**
Latency, tier-2 path	Improved — the local filter removes an estimated 30–40% of classifier calls (the 'thank you' / 'correct' family dominates the IGNORE bucket) at ~455 ms p50 each	**medium**
Tokens	~5–8% reduction (fewer tier-2 calls; answer volume unchanged)	**medium**
Accuracy — missed questions	12.5% → an estimated 4–6% after the D1 fix	**medium**

Dimension	Expected change	Confidence
Accuracy — unnecessary answers	11.3% → ~8% after tightening keyword_match	**medium**
Follow-ups	Unchanged — already correct in all 5 scenarios	**high**
Miscommunication	Unchanged — needs a session vocabulary, which is a separate change	**high**
Rate-limit pressure	Slightly reduced	**high**

Risks of the hybrid, stated plainly

- Raising tier-2's willingness to say ANSWER will raise false positives too. The gain is real but it is a trade, not a free win — measure both sides.
- The local non-content filter must be audio-aware like the existing `transcript_quality` stage, or it will drop a genuine spoken 'thank you, that's all' that ends the interview.
- Fixing D2 by removing `"can you explain"` from the follow-up set will make a genuine bare 'Can you explain?' wait 1.5 s instead of firing instantly. That is the correct trade, but it is a visible behaviour change.

24. Implementation Requirements

Listed for the decision, not implemented. Nothing in this section has been built.

#	Change	Where	Size
R1	Add an explicit non-content word-list check, audio-aware, as an early exit before the tier-2 classifier call	question_gate.py, in the tier-2 guard at ~line 348	small
R2	**D1** — rework the tier-2 prompt so IGNORE means 'definitely nothing' and unclear cases fall to WAIT, not IGNORE; add the indirect-request phrasings ('I wanted to ask about…', 'I'd like to know…', 'something about…') to `_QUESTION_PREFIXES`	question_gate.py:532 and :2201	medium
R3	**D2** — remove bare-verb openers ('can you explain', 'can you elaborate') from the exact follow-up set, or require them to end in '?' / terminal punctuation before they count as complete	question_gate.py:1649 `_is_followup_question()`	small
R4	**D3** — stop splitting clauses on a period that follows a single letter, so 'A.P.I.' and 'Node.js' survive; and do not let focus selection drop the clause containing the corrected term	question_gate.py:1252 `_CLAUSE_BOUNDARY`, :1302 `_select_focus()`	medium
R5	Tighten `keyword_match` so a well-formed statement naming a technology is not answered as a question about it (currently fires on 'I worked on a Django project.')	question_gate.py:1181 `_classify_clause()`	small
R6	*Separate from this decision:* build a session vocabulary from the resume/JD and pass it to STT and to the answer prompt, so 'Rack' resolves to RAG	stt/* prompt biasing + `_build_answer_messages()`	medium

25. Regression Tests Required Before Implementation

Run all of these **before** and **after** any change, and compare. The first four already have harnesses in the repo.

#	Test	Pass condition
T1	`client/test_question_gate_scenarios.py` — full offline suite	No scenario changes verdict, except the ones a fix deliberately targets

#	Test	Pass condition
T2	The 104-case rule-gate battery from this audit	No case that currently ANSWERs starts rejecting
T3	Incomplete-question timing matrix (3 questions × 4 pause lengths)	Every cell reads **1** answer — including the ‘Can you explain’ row, which currently reads 2
T4	Scenario-setup merge (SC6): setup sentence then question, 1–5 s apart	One answer containing both sentences
T5	Replay the 988 logged gate decisions through the new gate	Accept rate stays within 40–55%; no new accepts among the labelled noise set
T6	Re-label a fresh seeded sample of 80 accepts + 80 rejects	False negatives below 10%; false positives no higher than today's 11.3%
T7	Noise-safety test: send the 15 verbatim noise strings from section 4 through the full pipeline	**Zero** reach the answer model
T8	Live follow-up A/B: the 5 scenarios with real Groq calls	All 5 answer the right topic; the bare ‘Why?’ does not drift off topic
T9	Token/latency regression over one full mock interview	Answer calls per interview unchanged ±10%; first-token p95 no worse
T10	Barge-in: a new question over a streaming answer	The old answer is cancelled, exactly one new answer starts

Report generated 24 August 2026 from the working tree at commit adc8a75. All measurements were taken against unchanged production code, prompts and settings. No production file was modified during this audit.